Experiment No. 1

```
!pip install gensim

import gensim.downloader as api

model = api.load("glove-wiki-gigaword-100")

print("Word Relationships Using Vector Arithmetic:\n")

examples = [
   (["king", "woman"], ["man"], "king - man + woman"),
   (["paris", "italy"], ["france"], "paris - france + italy"),
   (["doctor", "woman"], ["man"], "doctor - man + woman")
]

for positive, negative, expression in examples:
   result = model.most_similar(positive=positive, negative=negative, topn=1)
   word, score = result[0]
   print(f"{expression} = {word} (similarity: {score:.3f})")
```

Experiment No. 2

```
import gensim.downloader as api
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

model = api.load("glove-wiki-gigaword-50")
words =
["computer","software","hardware","internet","data","network","algorithm","programming"
,"learning","robot"]
vectors = [model[word] for word in words]

pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(vectors)

plt.figure(figsize=(8,6))
for i, word in enumerate(words):
   x, y = reduced_vectors[i]
   plt.scatter(x, y)
   plt.text(x + 0.01, y + 0.01, word)
   plt.title("word embedding visualization using PCA")
   Plt.xlabel("component1")
   Plt.ylabel("component2")
plt.show()


input="computer"
similar_words=model.most_similar(input,topn=5)
print("Top 5 most similar words are:")
for word,score in similar_words:
    print(f"{word}→Similarity:{score:.3f}")
```

Experiment No. 3

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
import nltk

nltk.download('punkt', quiet=True)

corpus = [
   "patient is treated in hospital",
   "doctor prescribes medicine",
   "nurse cares for patient",
   "hospital provides medical treatment",
   "surgery is performed by surgeon",
   "medicine helps patient recover",
   "doctor and nurse work in hospital"
]

data = [word_tokenize(s.lower()) for s in corpus]

print("=== Tokenized Corpus ===")
for s in data:
   print(s)

model = Word2Vec(data, vector_size=50, window=3, min_count=1, sg=1)

print("\n=== Word2Vec Model Trained Successfully ===")

for word in ["patient", "doctor"]:
   print(f"\nSimilar words to '{word}':")
   for w, score in model.wv.most_similar(word, topn=5):
      print(f"  {w} --> similarity = {score:.3f}")

print("\n=== Word Similarity Scores ===")
print(f"doctor ↔ hospital : {model.wv.similarity('doctor','hospital'):.3f}")
print(f"doctor ↔ medicine : {model.wv.similarity('doctor','medicine'):.3f}")
```

Experiment No. 4

```
!pip install gensim transformers sentencepiece -q
import gensim.downloader as api
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

glove = api.load("glove-wiki-gigaword-100")
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = "Explain the role of Artificial Intelligence in healthcare."
keywords = ["artificial_intelligence", "healthcare"]
similar_words = []

for word in keywords:
  if word in word_model:
    similar = word_model.most_similar(word, topn=3)
    for w in similar:
      similar_words.append(w[0])

# Enriched prompt
enriched = prompt + "\nInclude topics like: " + ", ".join(words)

# Generate function
def generate(text):
  inputs = tokenizer(text, return_tensors="pt")
  output = model.generate(**inputs, max_length=200)
  return tokenizer.decode(output[0], skip_special_tokens=True)

# Responses
original_response = generate(prompt)
enriched_response = generate(enriched)

# Output
print("Original Prompt:\n", prompt)
print("\nSimilar Words:", words)

print("\n===== ORIGINAL RESPONSE =====")
print(original_response)

print("\n===== ENRICHED RESPONSE =====")
print(enriched_response)
```

```
print("\nLength Comparison")
print("Original :", len(original_response))
print("Enriched :", len(enriched_response))
```

Experiment No. 5

```
!pip install gensim -q
import gensim.downloader as api

print("Loading word embedding model...\n")
model = api.load("glove-wiki-gigaword-100")
print("Model loaded successfully!\n")

seed_word = input("Enter a seed word: ").lower()

print("\nTop 5 Similar Words:\n")

try:
   similar_words = model.most_similar(seed_word, topn=5)

   word_list = []

   for word, score in similar_words:
      print(word)
      word_list.append(word)

except:
   print("Word not found in vocabulary. Try another word.")
   exit()


print("\nGenerated Paragraph:\n")

paragraph = f"""
One day, a {seed_word} decided to start an adventure.
On the way, it met a {word_list[0]} and a {word_list[1]}.
They travelled across {word_list[2]} lands full of {word_list[3]} experiences.
In the end, their journey became a {word_list[4]} story that inspired everyone.
"""

print(paragraph)
```

Experiment No. 6

```
from transformers import pipeline

print("Loading Sentiment Model...\n")
sentiment = pipeline("sentiment-analysis")
print("Model Loaded Successfully\n")

reviews = [
   "The product quality is excellent.",
   "I am very unhappy with the service.",
   "Delivery was fast and packaging was good.",
   "The experience was terrible.",
   "It is okay, not bad."
]

results = sentiment(reviews)
print("Sentiment Analysis Results:\n")

for review, result in zip(reviews, results):
   print("Review:", review)
   print("Sentiment:", result['label'])
   print("Confidence:", round(result['score'], 3))
   print("-" * 40)
```

Experiment No. 7

```
!pip install transformers==4.41.2

from transformers import pipeline

print("Loading Summarization Model...")
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
print("Model Loaded Successfully!")

text = """
Artificial Intelligence (AI) is transforming industries such as healthcare,
education, finance, and transportation. AI systems analyze large amounts of data
to make intelligent decisions. Applications include chatbots, recommendation systems,
and autonomous vehicles. However, ethical concerns such as privacy and bias must
be addressed for responsible AI development.
"""

summary = summarizer(text, max_length=60, min_length=20, do_sample=False)

print("\nOriginal Text:\n", text)
print("\nSummarized Text:\n", summary[0]['summary_text'])
```

Experiment No. 8

```
!pip install langchain-cohere cohere -q

import os
from google.colab import drive
from langchain_cohere import ChatCohere

# API Key
os.environ["COHERE_API_KEY"] = "YOUR_API_KEY"

# Mount Drive
drive.mount('/content/drive')

# Read document
with open("/content/drive/My Drive/Colab Notebooks/document.txt", "r") as f:
   document = f.read()

# Load model
llm = ChatCohere(model="command-a-03-2025")

# Generate summary
response = llm.invoke(
   f"Read the document below and summarize it in 3 simple points:\n\n{document}"
)

print("Generated Summary:\n")
print(response.content)
```

Experiment No. 9

```
import os
from langchain_cohere import ChatCohere
from langchain_core.prompts import PromptTemplate
from langchain_community.utilities import WikipediaAPIWrapper
from langchain_community.tools import WikipediaQueryRun
from pydantic import BaseModel

os.environ["COHERE_API_KEY"] = "API KEY"

llm = ChatCohere(model="command-a-03-2025")

institution = input("Enter Institution Name: ")

wiki = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
text = wiki.run(institution)

class InstitutionInfo(BaseModel):
    founder: str
    founded_year: str
    branches: str
    employees: str
    summary: str

class InstitutionParser:
    def parse(self, text):
        text = text.replace("*", "").strip()   # Clean formatting

        data = {
            "founder": "N/A",
            "founded_year": "N/A",
            "branches": "N/A",
            "employees": "N/A",
            "summary": ""
        }

        lines = text.split("\n")
        summary_lines = []

        for line in lines:
            line = line.strip()

            if line.lower().startswith("founder"):
```

```
            data["founder"] = line.split(":",1)[-1].strip()

        elif line.lower().startswith("founded"):
            data["founded_year"] = line.split(":",1)[-1].strip()

        elif line.lower().startswith("branches"):
            data["branches"] = line.split(":",1)[-1].strip()

        elif line.lower().startswith("employees"):
            data["employees"] = line.split(":",1)[-1].strip()

        elif "summary" in line.lower():
            summary_lines.append(line.split(":",1)[-1].strip())

        elif summary_lines:
            summary_lines.append(line)

    data["summary"] = " ".join(summary_lines)

    return InstitutionInfo(**data)

prompt = PromptTemplate(
    template="""
Extract the following details from the text:

Founder:
Founded:
Branches:
Employees:
Summary (4 lines):

Text:
{text}

Return output in same format.
""",
    input_variables=["text"]
)

final_prompt = prompt.format(text=text)
response = llm.invoke(final_prompt)

parser = InstitutionParser()
```

```
result = parser.parse(response.content)

print("\n--- Institution Details ---\n")
print("Founder:", result.founder)
print("Founded Year:", result.founded_year)
print("Branches:", result.branches)
print("Employees:", result.employees)
print("Summary:", result.summary)
```

Experiment No. 10

```
!pip install -q PyPDF2 sentence-transformers faiss-cpu cohere

import PyPDF2
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
import cohere

from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
print("Uploaded file:", file_name)

def load_pdf(file_path):
    text = ""
    with open(file_path, "rb") as file:
        reader = PyPDF2.PdfReader(file)
        for page in reader.pages:
            text += page.extract_text()
    return text

ipc_text = load_pdf(file_name)
print("IPC Text Loaded Successfully!")

def split_text(text, chunk_size=1000):
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i+chunk_size])
    return chunks

chunks = split_text(ipc_text)[:2000]
print("Total Chunks Used:", len(chunks))

model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(chunks)
print("Embeddings Created!")

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))
```

```
print("FAISS Index Ready!")

co = cohere.Client("fQtPj0lfhBvAKjmI2EUaaV5ZbWrIHwZPCo5u2SVE")

def ask_ipc(question):
    q_embedding = model.encode([question])

    D, I = index.search(np.array(q_embedding), k=3)
    context = " ".join([chunks[i] for i in I[0]])

    response = co.chat(
        model="command-nightly",
        message=f"""
Answer the question based on the Indian Penal Code context.

Context:
{context}

Question:
{question}

Answer:
"""
    )

    return response.text.strip()

print("\nIPC Chatbot Ready! (type 'exit' to quit)\n")

while True:
    query = input("Ask IPC Question: ")

    if query.lower() == "exit":
        print(" Exiting Chatbot. Goodbye!")
        break

    print("\n Answer:", ask_ipc(query), "\n")
```